

Documentação do projeto AgroBasis

1 - O Sistema

1.1 - Contexto atual

O estado do Mato Grosso é o maior produtor de grãos do Brasil, porém, os produtores e tradings operam em um ambiente de **extrema volatilidade**. O preço final da saca de soja não depende apenas da colheita, mas de variáveis globais (como as bolsas internacionais), macroeconômicas (Câmbio) e logísticas (Frete e Portos). Atualmente, muitas decisões de venda são tomadas baseando-se em planilhas estáticas por meio da experiência de mercado, ou seja, com a ausência de um modelo de dados confiável, o que pode gerar perdas milionárias por falta de precisão na margem real.

1.2 - Descrição simplificada

O **AgroBasis** é uma plataforma de suporte à decisão especializada na comercialização de **commodities agrícolas**, com foco inicial nas cadeias produtivas de **soja e milho**. O principal produto do sistema não é o grão físico, mas a **informação de margem líquida em tempo real**, permitindo que produtores e gestores identifiquem o momento exato de máxima rentabilidade para a venda de sua safra.

A operação do sistema segue este fluxo de valor:

- **Ingestão de Dados Externos:** O sistema monitora e coleta continuamente cotações de bolsas de valores internacionais, principalmente à de Chicago (CBOT), taxas de câmbio em tempo real e tabelas de fretes logísticos das principais rotas de escoamento de Mato Grosso.
- **Cálculo da Paridade de Exportação:** Os dados globais são convertidos para a realidade local. O motor de cálculo deduz do preço bruto os custos de transporte, tributação estadual, taxas portuárias e o diferencial de base regional (*Basis*), entregando como resultado o valor líquido do grão posto na fazenda.
- **Análise Probabilística de Risco:** Para mitigar a incerteza, o sistema realiza simulações matemáticas sobre a volatilidade do dólar e do frete. Em vez de um número estático, o usuário recebe uma curva de probabilidades que indica o nível de segurança de sua margem de lucro em diversos cenários futuros.
- **Vigilância e Notificação Ativa:** A plataforma monitora os cálculos 24/7 confrontando-os com as metas de lucro do usuário. Ao detectar que o mercado atingiu o patamar desejado, o sistema dispara alertas imediatos para dispositivos móveis, eliminando o atraso humano na tomada de decisão comercial.

1.3 - Objetivos

1.3.1 - Objetivo Funcional

Prover uma plataforma que calcule em tempo real a **Paridade de Exportação** (preço real na fazenda), realize simulações de risco baseadas em probabilidades e automatize alertas de oportunidade de venda.

1.3.2 - Objetivo Organizacional

Reduzir a incerteza financeira, maximizar a margem de lucro por saca e mitigar riscos de mercado, permitindo que o gestor aja de forma proativa e não reativa às oscilações da Bolsa.

1.4 - Objetivos do Sistema

1.4.1 - O que o sistema faz:

- **Integração Automatizada:** Consome dados de fontes externas de forma autônoma.
- **Cálculo de Paridade:** Processa a equação financeira completa, incluindo descontos tributários e logísticos específicos da região.
- **Simulação de Cenários:** Executa cálculos estatísticos para prever variações de margem baseadas na volatilidade dos insumos.
- **Comunicação de Oportunidades:** Notifica os stakeholders via canais de mensageria quando os alvos são atingidos.
- **Gestão de Gatilhos:** Permite que o usuário configure alvos/metast de preço e lucro.
- **Visualização de Dados:** Apresenta painéis com o histórico de preços e as curvas de probabilidade de lucro.

1.4.2 - O que o sistema não faz:

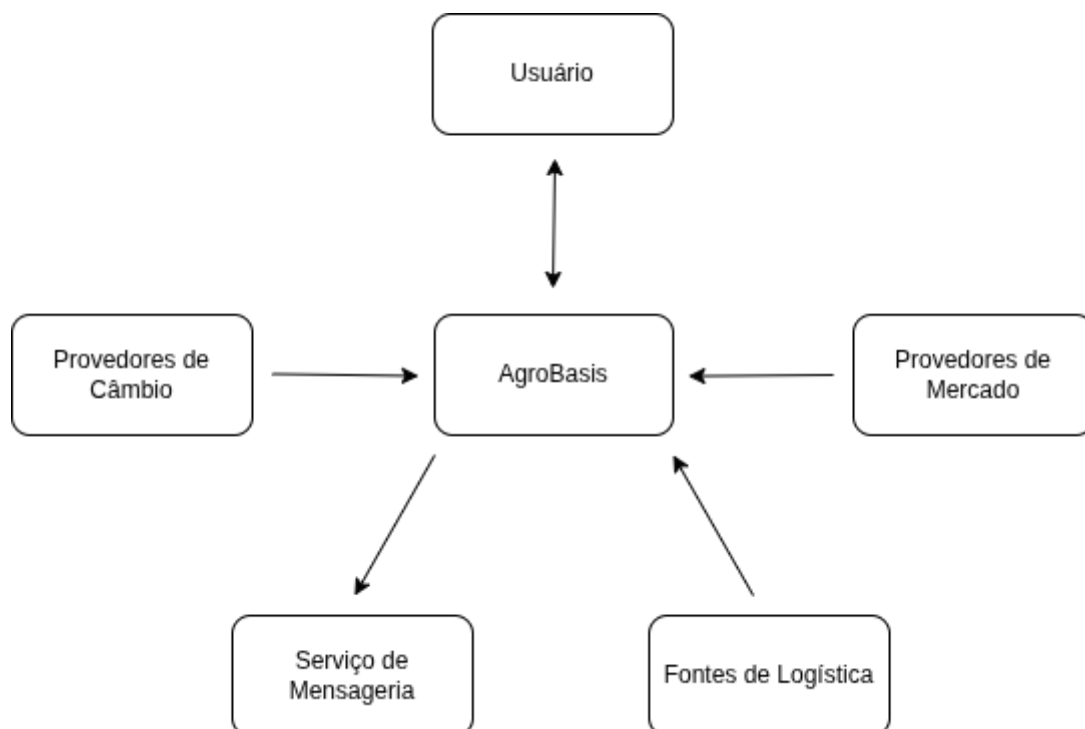
- **Execução de Trades:** O sistema não realiza a compra ou venda de ativos diretamente nas bolsas ou com tradings; ele é uma ferramenta de apoio à decisão, não uma plataforma de corretagem.
- **Gestão de Estoque:** Não controla o volume físico armazenado em silos ou armazéns.
- **Monitoramento de Safra:** Não utiliza imagens de satélite ou sensores de campo para prever produtividade agrícola.
- **Contabilidade e Fiscal:** Não gera notas fiscais ou relatórios contábeis para o fisco.
- **Consultoria Financeira:** O sistema apresenta dados e probabilidades, mas não emite recomendações subjetivas de investimento.

1.5 - Modelo de contexto

O modelo de contexto permite entender e visualizar a relação do sistema AgroBasis com os sistemas externos que fornecerão dados e recursos para o funcionamento geral do sistema como um todo.

Entidades Externas

1. **Usuário:** Interage com o sistema para definir metas de lucro, consultar preços e receber alertas.
2. **Provedores de Mercado:** Serviços externos que fornecem o preço bruto da commodity em Chicago ou São Paulo.
3. **Provedores de Câmbio:** Serviços externos que fornecem a cotação em tempo real do Dólar.
4. **Fontes de Logística:** Sistemas que fornecem o custo do transporte por tonelada em rotas específicas do MT.
5. **Serviço de Mensageria:** O canal externo usado para empurrar as notificações para o celular do usuário.



1.6 - Implicações morais, éticas e de privacidade

Esta seção define o compromisso do **AgroBasis** com a segurança das informações e a transparência das análises fornecidas ao produtor.

1.6.1 - Confidencialidade Estratégica

No agronegócio, o custo de produção e a estratégia de venda são os maiores diferenciais de uma empresa.

- **O Compromisso:** O sistema garante que os dados de uma fazenda jamais sejam misturados ou fiquem visíveis para outra. Cada empresa possui seu próprio centro de dados isolado.

- **Na Prática:** Mesmo que o sistema use médias de mercado para ajudar na decisão, ele nunca usará o dado privado de um produtor para dar uma vantagem ao concorrente dele.

1.6.2 - Governança de Dados e LGPD (Direito à Revogação)

O **AgroBasis** é construído sob as regras da Lei Geral de Proteção de Dados (LGPD), tratando as informações com o rigor que a legislação brasileira exige.

- **Segurança na Prática:** Utilizamos barreiras digitais e chaves de acesso para garantir que apenas pessoas autorizadas pela empresa possam visualizar os dados financeiros. É como ter um sistema de câmeras e alarmes, mas voltado para os dados da sua colheita.
- **Direito de Propriedade:** Os dados pertencem ao usuário. A qualquer momento, o gestor pode solicitar:
 - **Acesso:** Ver tudo o que o sistema sabe sobre sua operação.
 - **Correção:** Ajustar informações de custos ou metas.
 - **Revogação:** Solicitar a exclusão total de seus dados do sistema caso decida encerrar a parceria.

1.6.3 - Transparência e Responsabilidade na Decisão

O sistema é um assistente de alta precisão, mas a palavra final é sempre do gestor.

- **O Compromisso:** Toda predição ou cálculo de lucro será acompanhado de uma explicação simples sobre quais fatores levaram àquele resultado.
- **Na Prática:** O sistema não é uma "caixa preta". Ele mostra claramente de onde veio cada informação e qual a "idade" daquele dado (se é um preço de agora ou de 10 minutos atrás), para que o produtor não tome decisões baseadas em informações defasadas.

1.6.4 - Ética na Informação Externa

Dependemos de informações vindas de mercados globais e empresas de logística.

- **O Compromisso:** Se uma fonte de dados externa, como o preço da bolsa, falhar ou sofrer atrasos, o sistema avisará o usuário imediatamente.
- **Na Prática:** É preferível o sistema dizer "Não tenho a cotação exata agora" do que mostrar um preço errado que possa induzir o produtor a um prejuízo financeiro.

1.7 - Estudo de Viabilidade Técnica

1.7.1 Avaliação da Stack Tecnológica

A combinação de tecnologias proposta é altamente viável e segue os padrões de mercado para sistemas financeiros e de alta disponibilidade.

- **Núcleo de Processamento (Java):** A escolha do Java para o motor de paridade e orquestração é tecnicamente robusta, pois a linguagem oferece estabilidade,

tipagem forte para cálculos financeiros e excelente gerenciamento de concorrência com o uso de Virtual Threads, o que é essencial para processar múltiplas simulações simultaneamente.

- **Inteligência de Dados (Python):** A integração com Python para os modelos de predição é o caminho mais prático devido à maturidade das bibliotecas de ciência de dados. O uso de suas bibliotecas especializadas permite que essa camada funcione como um microserviço de baixa latência, comunicando-se de forma eficiente com o núcleo em Java.
- **Persistência e Cache (PostgreSQL e Redis):** O uso de PostgreSQL garante a integridade dos dados históricos, enquanto o Redis resolve o gargalo de performance ao evitar requisições repetitivas a APIs de mercado, armazenando cotações voláteis em memória.

1.7.2 Disponibilidade e Integração de Dados

A viabilidade do AgroBasis depende diretamente do acesso a dados externos, o que representa o ponto de maior atenção no estudo.

- **Dados de Mercado (Bolsas e Câmbio):** Existem diversas APIs comerciais, como Barchart ou Alpha Vantage, que fornecem cotações da CBOT e taxas de câmbio. O custo dessas APIs em um ambiente de produção deve ser considerado, mas para o desenvolvimento, existem planos gratuitos ou de baixo custo que validam a viabilidade técnica.
- **Dados Logísticos e Regionais:** O acesso a dados de frete de Mato Grosso pode exigir a construção de scripts ou integradores para fontes como o IMEA, uma tarefa tecnicamente comum mas que exige manutenção constante devido a possíveis mudanças estruturais nos sites de origem.

1.7.3 Desafios de Processamento e Performance

- **Simulações de preço:** Ao utilizar a simulação Monte Carlo, executar dez mil ou mais iterações de cenários probabilísticos exige poder computacional. A viabilidade é confirmada pelo uso do Java moderno, que permite a execução paralela dessas tarefas sem comprometer a resposta da interface do usuário.
- **Sistemas Distribuídos e Mensageria:** O uso de Kafka ou RabbitMQ para a comunicação entre módulos resolve o problema de acoplamento, permitindo que o sistema continue funcionando mesmo que um dos serviços sofra uma instabilidade temporária. Isso aumenta a resiliência global da arquitetura.

1.7.4 Complexidade de Implementação

A curva de aprendizado para integrar duas linguagens diferentes e configurar um ambiente distribuído é o principal desafio de engenharia. No entanto, o uso de **Docker** para a containerização de todos os serviços mitiga o risco de problemas de ambiente, garantindo que o comportamento do sistema seja idêntico em desenvolvimento e produção.

Manter a comunicação entre os componentes de forma limpa é algo a ser considerado durante todo processo de engenharia do projeto, pois irá facilitar o desenvolvimento e organização do código.

1.7.5 Matriz de Riscos Técnicos

Risco	Impacto	Mitigação
Latência na API de Mercado	Alto	Implementação de Caching agressivo com Redis e estratégias de <i>fallback</i> para dados históricos.
Inconsistência de Tipagem	Médio	Uso estrito de <code>BigDecimal</code> no Java e validação de schemas JSON entre Java e Python.
Escalabilidade da Mensageria	Baixo	Uso de instâncias gerenciadas ou bem configuradas de RabbitMQ para evitar perda de notificações.
Mudança de Regras Tributárias	Médio	Implementação do <i>Strategy Pattern</i> no Java para facilitar a atualização de fórmulas de impostos sem alterar o core.

1.7.6 - Parecer de Viabilidade

O projeto **AgroBasis** é considerado **tecnicamente viável**. A stack escolhida é equilibrada e capaz de suportar os requisitos de performance e precisão exigidos pelo setor de agronegócio. O maior esforço técnico residirá na integração harmônica entre o motor de cálculo em Java e os modelos preditivos em Python, além da garantia de resiliência nas comunicações distribuídas.

1.8 - Processos de Desenvolvimento do Sistema

1.8.1 - Processos:

1. **Concepção e Alinhamento:** Estudo do mercado de Mato Grosso e definição da ideia central.
2. **Especificação e Dimensionamento:** Definição dos RFs e RNFs e escolha das tecnologias de baixo custo.
3. **Estratégia de Desenvolvimento:** Configuração do ambiente e planejamento das funcionalidades prioritárias.

4. **Desenvolvimento por Funcionalidade:** Ciclo repetitivo de escrita de testes (TDD) e codificação das funções do sistema.
5. **Implantação e Estabilização:** Publicação do sistema em servidor e testes reais com dados de mercado para garantir que nada trave.
6. **Sustentação e Evolução:** Monitoramento da saúde das APIs externas e adição de novos grãos ou rotas logísticas conforme a demanda.

1.8.2 - Eixos de Trabalho:

- **Gestão de Projetos:** Organização das tarefas por "funcionalidades" para garantir o fluxo de entrega.
- **Engenharia de Software:** Aplicação de padrões de projeto e TDD para garantir um código limpo e fácil de manter.
- **Gestão de Segurança:** Foco no isolamento de dados entre empresas e proteção de segredos comerciais.
- **Gestão de Infraestrutura:** Foco na economia de recursos para rodar o sistema em servidores baratos.
- **Produção de Artefatos:** Criação de documentação para que o sistema não seja uma "caixa preta".
- **Alinhamento de Domínio:** Garantia de que o software fala a língua do produtor e das tradings de MT.
- **Gestão de Dados:** Monitoramento da qualidade e do horário das informações vindas das bolsas e fretes.
- **Evolução Tecnológica:** Planejamento para que o sistema possa crescer sem precisar ser reescrito do zero.

1.9 - Metodologia de Desenvolvimento

A metodologia adotada para a execução do projeto fundamenta-se na união do **Desenvolvimento Focado em Funcionalidade (FDD - Feature-Driven Development)** com o rigor técnico do **Desenvolvimento Orientado a Testes (TDD - Test-Driven Development)**. Esta abordagem visa garantir a entrega incremental de valor aliada à integridade matemática necessária para operações de mercado, com foco em implementar uma solução e depois refatorar.

1.9.1 Desenvolvimento Focado em Funcionalidade (FDD)

O processo de construção é estruturado a partir da decomposição do sistema em funcionalidades discretas e acionáveis. Cada etapa de desenvolvimento concentra-se em uma "entrega de valor" específica, permitindo o acompanhamento claro do progresso do projeto.

- **Justificativa:** Esta técnica assegura que o sistema seja funcional desde as suas etapas iniciais, facilitando a validação de módulos críticos, como o cálculo de paridade, antes da implementação de módulos secundários.

1.9.2 Desenvolvimento Orientado a Testes (TDD)

A técnica de TDD é aplicada de forma transversal a todo o ciclo de codificação, seguindo o ciclo **Red-Green-Refactor**:

1. **Red (Falha)**: Escrita de um teste automatizado para uma regra de negócio específica antes da implementação da lógica.
 2. **Green (Sucesso)**: Implementação do código mínimo necessário para a aprovação do teste.
 3. **Refactor (Otimização)**: Melhoria da estrutura do código para garantir legibilidade e performance, sem alteração do comportamento validado.
- **Justificativa**: Dada a natureza financeira do AgroBasis, o TDD atua como uma barreira contra erros de cálculo e regressões, garantindo que as fórmulas de lucro e risco permaneçam íntegras durante a evolução do sistema.

1.9.3 Refatoração

Utilizaremos uma técnica comum do desenvolvimento ágil, que se dá da seguinte maneira para toda funcionalidade em foco:

1. Criação das soluções para determinadas dificuldades da funcionalidade até que ela funcione na prática
2. Análise do código
3. Refatoração de cada bloco de código, se adequando aos padrões do projeto e separando as responsabilidades

2 - O Sistema

2.1 - Análise de Mercado

O mercado de tecnologia para o agronegócio no Brasil está em plena expansão, mas ainda existe um abismo entre o que acontece na Bolsa de Valores e o que chega no celular do produtor em Mato Grosso.

- **Público-Alvo**: Tradings de médio porte, grandes produtores rurais de grãos, principalmente soja e milho, e gestores de cooperativas em cidades como Sorriso, Lucas do Rio Verde e Rondonópolis.
- **A Oportunidade**: Mato Grosso é o maior produtor nacional, mas a maioria das decisões ainda é tomada exclusivamente baseando-se em experiência ou em planilhas que não conversam com o mercado em tempo real. Existe uma demanda reprimida por ferramentas que simplifiquem a logística complexa da região (fretes caros e rotas variáveis).

2.2. Trabalhos Relacionados (O que já existe)

Para o AgroBasis ser relevante, ele precisa saber quem são os "vizinhos" no mercado:

- **Sistemas de Tradings Próprias**: Grandes empresas como Amaggi ou Cargill possuem sistemas internos, mas eles são fechados e focados no lucro da própria empresa, não do produtor.

- **Plataformas de Informação (Ex: IMEA):** O Instituto Mato-grossense de Economia Agropecuária fornece dados excelentes, mas são estáticos. Eles dão o dado, mas não a ferramenta de simulação para o caso específico do usuário.
- **Terminais de Mercado:** São ferramentas poderosíssimas, mas extremamente caras e muito complexas para o produtor rural comum.

2.3 - Grau de Inovação

O AgroBasis possui grau de inovação alto.

1. **Regionalismo Prático:** Diferente de sistemas globais, o AgroBasis considera de forma específica, os parâmetros da região Mato-Grossense. Por exemplo, ele considera o custo real do frete da BR-163 e os impostos locais como a forma padrão dos custos.
2. **Democratização da Probabilidade:** Ele traz uma matemática pesada para uma interface simples. O usuário não precisa ser um estatístico para entender uma informação gerada.
3. **Vigilância Ativa:** A maioria dos sistemas atuais são passivos e apenas geram a informação. O AgroBasis se preocupa também com a entrega ao usuário, ele vigia o mercado para o usuário e o avisa proativamente.

3 - Requisitos

3.1 Elicitação de requisitos

3.1.1 Requisitos Funcionais

- **RF01** - O sistema deve permitir a criação de contas empresariais com múltiplos usuários vinculados, utilizando autenticação para proteger dados privados.
- **RF02** - O sistema deve permitir que o administrador da empresa defina níveis de visibilidade (quem pode ver custos reais, quem pode configurar alertas, etc.).
- **RF03** - O sistema deve permitir que o usuário insira variáveis de custo específicas (taxas bancárias, corretagens e custos administrativos).
- **RF04** - O sistema deve permitir que cada empresa selecione quais commodities deseja monitorar em seu painel principal.
- **RF05** - O sistema deve permitir a inserção e atualização do volume de grãos disponível em estoque para comercialização.
- **RF06** - O sistema deve calcular o lucro real permitindo que o usuário insira seus próprios custos de produção e armazenagem.
- **RF07** - O sistema deve permitir a inserção de índices de umidade e impureza para aplicar descontos automáticos no peso líquido da carga, conforme padrões oficiais.
- **RF08** - O sistema deve utilizar custos de frete simulados por padrão, mas permitir a substituição por valores reais inseridos pelo usuário.
- **RF09** - O sistema deve executar modelos matemáticos para gerar intervalos de confiança e probabilidades de lucro baseados na volatilidade do mercado.
- **RF10** - O sistema deve oferecer a opção de comparar os indicadores da própria empresa com a média histórica e atual da região.

- **RF11** - O sistema deve permitir a configuração de limites personalizados de preço e lucro para o disparo de notificações automáticas via mensageria.
- **RF12** - O sistema deve apresentar tendências históricas e projeções de custos de transporte para as rotas selecionadas em Mato Grosso.
- **RF13** - O sistema deve exibir em uma única interface o resumo do mercado (Chicago, Dólar e Frete Local) e a situação da empresa.
- **RF14** - O sistema deve calcular o valor total da safra em estoque ao longo do tempo, permitindo comparar a valorização do estoque atual contra períodos passados.
- **RF15** - O sistema deve informar a fonte original e o horário da última atualização para cada indicador exibido.
- **RF16** - O sistema deve gerar documentos em PDF com gráficos e cálculos de paridade para apresentações externas e arquivo.
- **RF17** - O sistema deve logar as condições exatas do mercado no momento em que um gatilho de preço configurado pelo usuário foi atingido.

3.1.2 Requisitos Não Funcionais

- **RNF1** - Isolamento lógico rigoroso entre os dados de diferentes organizações, garantindo que usuários de uma empresa jamais acessem informações de outra.
- **RNF2** - Criptografia de dados sensíveis identificados como segredo de negócio, especificamente custos de produção, margens de lucro e metas de preço, tanto em repouso quanto em trânsito.
- **RNF3** - Autenticação segura e moderna utilizando padrões que não gerem custos adicionais com serviços externos de terceiros.
- **RNF4** - Utilização de aritmética de alta precisão decimal em todos os motores de cálculo para evitar erros acumulados de arredondamento em grandes volumes financeiros.
- **RNF5** - Conclusão de simulações estatísticas de risco e probabilidade em um tempo máximo de 60 segundos, mantendo o feedback visual de progresso ao usuário.
- **RNF6** - Otimização do consumo de recursos para garantir a operação estável em servidores de entrada e baixo custo de manutenção.
- **RNF7** - Rastreabilidade obrigatória de dados, exibindo a fonte original e o horário da última atualização para cada cotação ou índice de mercado apresentado.
- **RNF8** - Persistência e mecanismo de retransmissão automática de notificações para assegurar a entrega de alertas mesmo em cenários de instabilidade de conexão.
- **RNF9** - Resiliência no cálculo através de um sistema de reserva, utilizando médias regionais automaticamente caso o perfil da empresa não possua dados próprios preenchidos.
- **RNF10** - Interface responsiva com foco prioritário em dispositivos móveis (Mobile-First), adaptada para uso em ambientes de campo e sob luz solar.
- **RNF11** - Baixa carga cognitiva no design dos dashboards, permitindo que o gestor compreenda a situação do mercado e da empresa em menos de 5 segundos.
- **RNF12** - Disponibilidade total via navegadores web modernos, eliminando a necessidade de instalação de softwares locais ou aplicativos específicos.
- **RNF13** - Saneamento e validação estrita de dados de entrada para impedir a inserção de valores financeiros incoerentes ou negativos.

- **RNF14** - Comunicação imediata de falhas de serviços externos, avisando o usuário quando uma fonte de dados estiver indisponível.
- **RNF15** - Latência máxima de 15 minutos para dados de mercado internacional e câmbio, respeitando os limites das fontes de dados gratuitas ou de baixo custo.

4 - Projeto do Sistema

4.1 - Modelo de Casos de Uso

ID: UC01 – Autenticar Usuário

- **Ator Principal:** Gestor Comercial / Administrador.
- **Pré-condições:** O usuário deve possuir uma conta previamente vinculada a uma organização.
- **Fluxo Principal:**
 - O usuário acessa a interface de entrada do sistema.
 - O sistema solicita as credenciais de acesso.
 - O sistema valida as credenciais contra a base de dados da organização.
 - O sistema concede o acesso e direciona o usuário ao dashboard principal.
- **Fluxos Alternativos / Exceções:**
 - Caso as credenciais sejam inválidas, o sistema exibe uma mensagem de erro e solicita nova tentativa.
 - Após sucessivas tentativas falhas, o sistema bloqueia temporariamente o acesso por questões de segurança.

ID: UC02 – Consultar Paridade

- **Ator Principal:** Gestor Comercial.
- **Pré-condições:** O sistema deve ter realizado a captura recente das cotações de bolsa e câmbio.
- **Fluxo Principal:**
 - O usuário acessa a funcionalidade de consulta de paridade.
 - O sistema identifica a localização da fazenda e a commodity padrão.
 - O sistema calcula o preço líquido subtraindo fretes e tributos da cotação convertida.
 - O sistema exibe o valor final por saca de forma destacada.
- **Fluxos Alternativos / Exceções:**
 - Caso o usuário não tenha cadastrado fretes próprios, o sistema aplica o custo da rota logística simulada.
 - Se a cotação de mercado estiver indisponível, o sistema exibe o último valor armazenado com um aviso de dado defasado.

ID: UC03 – Realizar Simulação de Risco

- **Ator Principal:** Gestor Comercial.
- **Pré-condições:** Caso de Uso UC02 concluído para fornecer a base de cálculo.
- **Fluxo Principal:**

- O usuário solicita a execução de uma simulação de risco para uma carga específica.
- O usuário pode ajustar variáveis dentro de intervalos realistas definidos pelo sistema.
- O sistema executa o modelo probabilístico.
- O sistema exibe a curva de probabilidade de lucro e o nível de confiança para a venda futura.
- **Fluxos Alternativos / Exceções:**
 - Se o usuário inserir um valor fora do intervalo de segurança, o sistema bloqueia a entrada e sugere o valor máximo permitido para manter o realismo da análise.

ID: UC04 – Configurar Gatilhos de Alerta

- **Ator Principal:** Gestor Comercial.
- **Pré-condições:** Usuário autenticado e organização configurada.
- **Fluxo Principal:**
 - O usuário define um parâmetro de alvo (ex: Preço da saca > R\$ 150,00).
 - O sistema registra o gatilho e inicia o monitoramento contínuo do mercado.
 - Ao atingir o valor alvo, o sistema gera uma notificação acionável via mensageria.
 - A notificação contém um link direto para a análise detalhada no sistema.
- **Fluxos Alternativos / Exceções:**
 - O usuário pode desativar ou editar o gatilho a qualquer momento na central de alertas.

ID: UC05 – Cadastrar Custos Operacionais

- **Ator Principal:** Gestor Comercial.
- **Pré-condições:** Acesso ao módulo de configurações da empresa.
- **Fluxo Principal:**
 - O usuário insere dados privados (custo de produção, armazenagem diária, taxas de corretagem).
 - O sistema valida o formato dos dados e armazena de forma criptografada.
 - Os novos custos passam a ser priorizados nos cálculos de paridade futuros daquela empresa.
- **Fluxos Alternativos / Exceções:**
 - O sistema permite a limpeza dos dados, retornando ao uso de médias regionais automáticas.

ID: UC06 – Visualizar Dashboard de Performance

- **Ator Principal:** Gestor Comercial.
- **Pré-condições:** Existência de histórico de cotações e estoque declarado.
- **Fluxo Principal:**
 - O sistema consolida os indicadores de mercado em painéis gráficos.
 - O sistema apresenta a valorização do estoque atual comparada a períodos passados.

- O usuário navega entre diferentes commodities através de filtros.
- **Fluxos Alternativos / Exceções:**
 - Caso faltem dados de estoque, o dashboard exibe apenas os indicadores de mercado regionais.

ID: UC07 – Exportar Relatórios de Viabilidade

- **Ator Principal:** Gestor Comercial.
- **Pré-condições:** Visualização de uma análise ou dashboard ativa.
- **Fluxo Principal:**
 - O usuário aciona o comando de exportação.
 - O sistema gera um arquivo em formato PDF contendo os gráficos e as tabelas de paridade vigentes.
 - O sistema disponibiliza o download imediato do documento.
- **Fluxos Alternativos / Exceções:**
 - Em caso de erro na geração do arquivo, o sistema solicita que o usuário tente novamente ou verifique a conexão.

ID: UC08 – Gerenciar Perfis e Permissões

- **Ator Principal:** Administrador da Empresa.
- **Pré-condições:** Usuário logado com perfil de nível administrativo.
- **Fluxo Principal:**
 - O administrador acessa a lista de usuários vinculados à organização.
 - O administrador define permissões de leitura ou escrita para módulos sensíveis.
 - O sistema atualiza as restrições de acesso em tempo real.
- **Fluxos Alternativos / Exceções:**
 - O administrador não pode remover sua própria permissão de acesso para evitar o bloqueio total da conta empresarial.

4.2 - Arquitetura do Sistema

Esta é a seção técnica definitiva que descreve a estrutura interna e o ecossistema tecnológico do **AgroBasis**. O projeto baseia-se em uma arquitetura de serviços distribuídos, projetada para ser resiliente, modular e financeiramente sustentável. A arquitetura do AgroBasis é dividida em unidades lógicas independentes que colaboram entre si para transformar dados brutos de mercado em inteligência para o produtor. A separação de responsabilidades garante que o sistema suporte o processamento pesado de simulações sem comprometer a fluidez da interface do usuário.

4.2.1 - Containers de Aplicação e Serviços

- **Interface de Alta Fidelidade (Frontend):**
Construída com bibliotecas modernas de JavaScript, como o React, a interface foca na experiência do usuário sob condições de campo. Ela é responsável pela renderização de gráficos complexos de tendência e dashboards de paridade. A tecnologia escolhida permite o desenvolvimento de uma aplicação web progressiva

(PWA), garantindo que o sistema seja responsivo e performático em navegadores mobile.

- **Gateway de Comunicação (API Gateway):**
Atuando como a sentinela do sistema, utiliza tecnologias como **Nginx** ou **Spring Cloud Gateway**. Ele centraliza as requisições do frontend e as distribui para os serviços internos. Suas funções incluem o gerenciamento de segurança (autenticação), o balanceamento de carga e a proteção contra ataques externos, garantindo que o núcleo do sistema nunca fique exposto diretamente.
- **Serviço de Gestão e Regras de Negócio (Core Service):**
Desenvolvido em **Java** com o framework **Spring Boot**, este é o motor principal do AgroBasis. O Java foi escolhido por sua robustez no tratamento de regras de negócio complexas e sua excelente gestão de concorrência. Este serviço gerencia o ciclo de vida do usuário, as permissões de acesso e o cálculo de paridade em tempo real. Para cumprir o requisito de precisão financeira, utiliza a classe **BigDecimal** para evitar erros de arredondamento em fórmulas utilizadas.
- **Serviço de Inteligência e Predição (Intelligence Service):**
Implementado em **Python** utilizando o framework **FastAPI**, este componente é especializado em ciência de dados. O Python é a escolha ideal pela maturidade de suas bibliotecas estatísticas (como **NumPy** e **Pandas**). Este serviço isola o processamento intensivo necessário para rodar as 10.000 iterações da Simulação de Monte Carlo, garantindo que o alto consumo de CPU desse processo não afete a velocidade de navegação no restante da plataforma.
- **Orquestrador de Mensagens (Message Broker):**
Utilizando o **Redis** ou **RabbitMQ**, este container funciona como o sistema de correio interno. Ele permite a comunicação assíncrona: quando o serviço em Java solicita uma simulação pesada, ele envia uma mensagem para a fila. O serviço em Python consome essa mensagem no seu tempo de processamento e devolve o resultado. Isso evita "travamentos" no sistema e garante que o usuário receba sua resposta assim que o cálculo for concluído.

4.2.2 - Camada de Persistência e Cache

- **Banco de Dados Centralizado (PostgreSQL):**
O PostgreSQL atua como o repositório oficial de dados relacionais. Escolhido por sua confiabilidade e conformidade com os padrões ACID, ele armazena perfis de usuários, históricos de estoque e configurações de custos reais de forma criptografada. Para manter o custo baixo e o isolamento alto, o banco é organizado em **Schemas** (espaços lógicos separados), onde o serviço Java e o serviço Python acessam apenas suas respectivas tabelas.
- **Repositório de Dados Voláteis (Redis Cache):**
Além de servir como broker de mensagens, o Redis é utilizado como banco de dados em memória para cache. Ele armazena as cotações das bolsas e do dólar capturadas nas últimas janelas de tempo. Isso reduz drasticamente a latência, pois o sistema não precisa "bater" no banco de dados principal ou em APIs externas toda vez que o usuário atualiza sua tela de paridade.

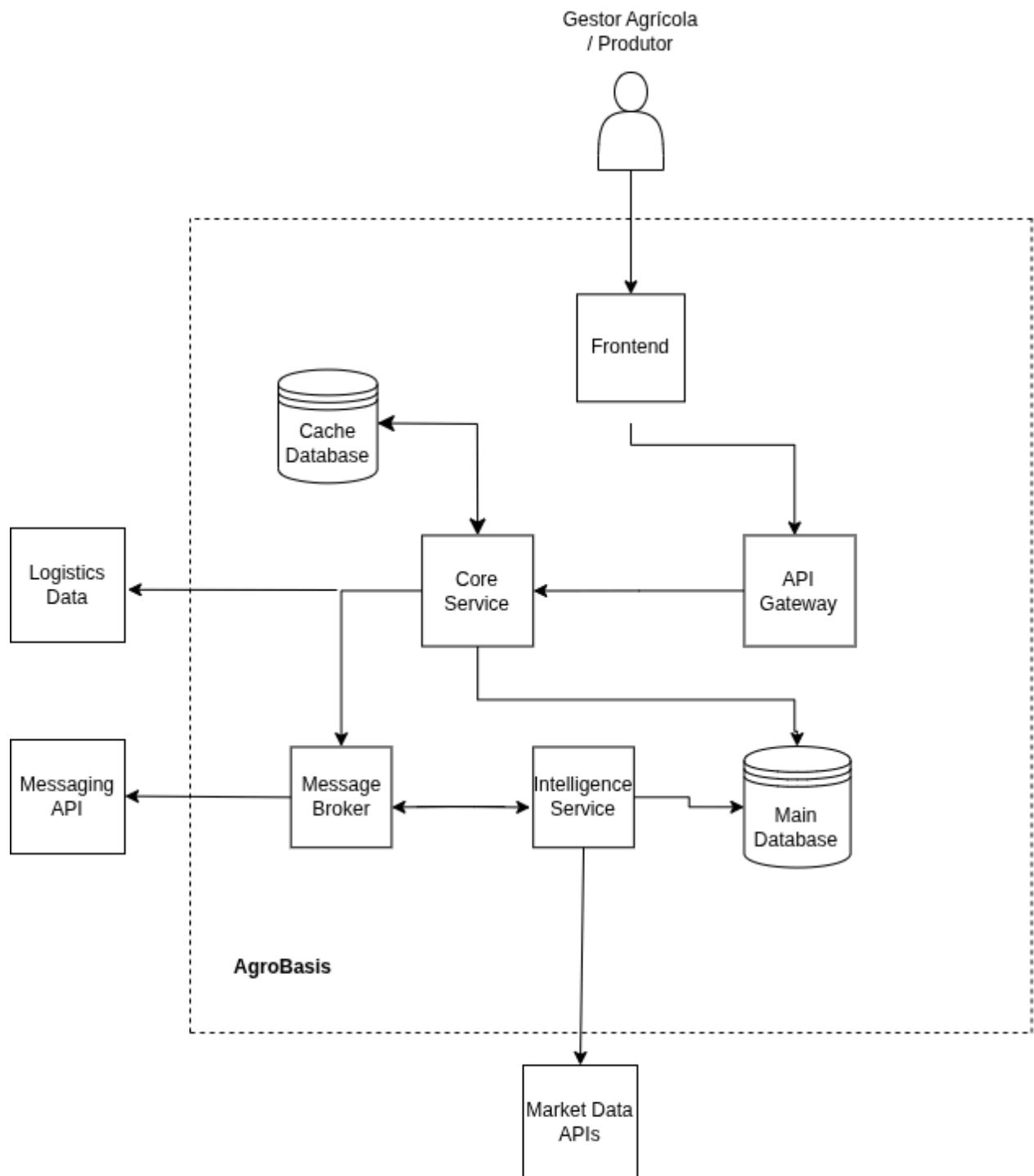
4.2.3 - Estratégia de Integração e Fluxo de Dados

A eficiência do AgroBasis reside na forma como esses componentes trocam informações:

- **Contratos JSON:** Toda a comunicação entre os serviços Java, Python e a interface web é feita via pacotes **JSON** padronizados. Isso garante que as diferentes tecnologias falem a mesma língua de forma leve e rápida.
- **Processamento Assíncrono:** Essencial para a resiliência. Tarefas que levam mais de alguns segundos (como simulações e envios de alertas via WhatsApp) nunca bloqueiam o sistema. Elas são colocadas em filas e processadas em segundo plano, respeitando a capacidade do servidor.

Componente	Tecnologia Sugerida	Justificativa de Engenharia
Backend Core	Java / Spring Boot	Estabilidade, segurança e alta performance em regras de negócio financeiras.
Inteligência	Python / FastAPI	Liderança em bibliotecas de estatística e processamento de dados científico.
Persistência	PostgreSQL	Robustez, suporte a schemas e alta integridade de dados relacionais.
Cache/Fila	Redis	Extrema velocidade para dados voláteis e orquestração de mensagens leves.
Comunicação	REST / JSON	Padrão de mercado para integração simples entre diferentes linguagens.

4.2.4 - Representação visual



4.3 - Diagrama Entidade-Relacionamento

Esta seção descreve a estrutura lógica de persistência do **AgroBasis**, detalhando como os dados são organizados no banco de dados relacional **PostgreSQL** para suportar as regras de negócio, a segurança da informação e o motor de simulação estatística.

5 - Descrição do Modelo de Entidade-Relacionamento (DER)

O banco de dados foi projetado sob a ótica da escalabilidade e do isolamento, utilizando **schemas** para separar dados de gestão de dados de mercado e processamento.

5.1 Núcleo Organizacional e Gestão de Acessos

A entidade central do sistema é a **Organization** (Empresa). Ela atua como a raiz de todos os dados privados.

- **Isolamento Multitenancy:** A relação entre **Organization** e **User** (1:N) garante que cada usuário esteja estritamente vinculado a uma empresa. O campo **organization_id** é propagado por quase todas as tabelas do sistema, servindo como a chave de filtragem primária para impedir o vazamento de dados entre diferentes empresas.
- **Commodity:** Uma tabela de referência que padroniza os grãos monitorados (Soja, Milho), garantindo que as cotações e o estoque sigam a mesma unidade de medida em todo o sistema.

5.2 Segurança e Segredo de Negócio

Para atender aos Requisitos Não-Funcionais de segurança, as tabelas de custos e estoque possuem tratamentos especiais:

- **Corporate_Costs (Custos Reais):** Esta tabela armazena o "segredo de negócio" do produtor. Diferente de outros campos, os valores de custo de produção e armazenagem são armazenados como **Blobs Criptografados**. Isso significa que, mesmo em caso de acesso indevido ao banco de dados, os valores financeiros permanecem ilegíveis sem as chaves de descryptografia gerenciadas pelo serviço Java.
- **Stock (Estoque):** Registra a quantidade física de grãos e seus índices de qualidade (umidade e impureza), essenciais para o cálculo de descontos na paridade final.

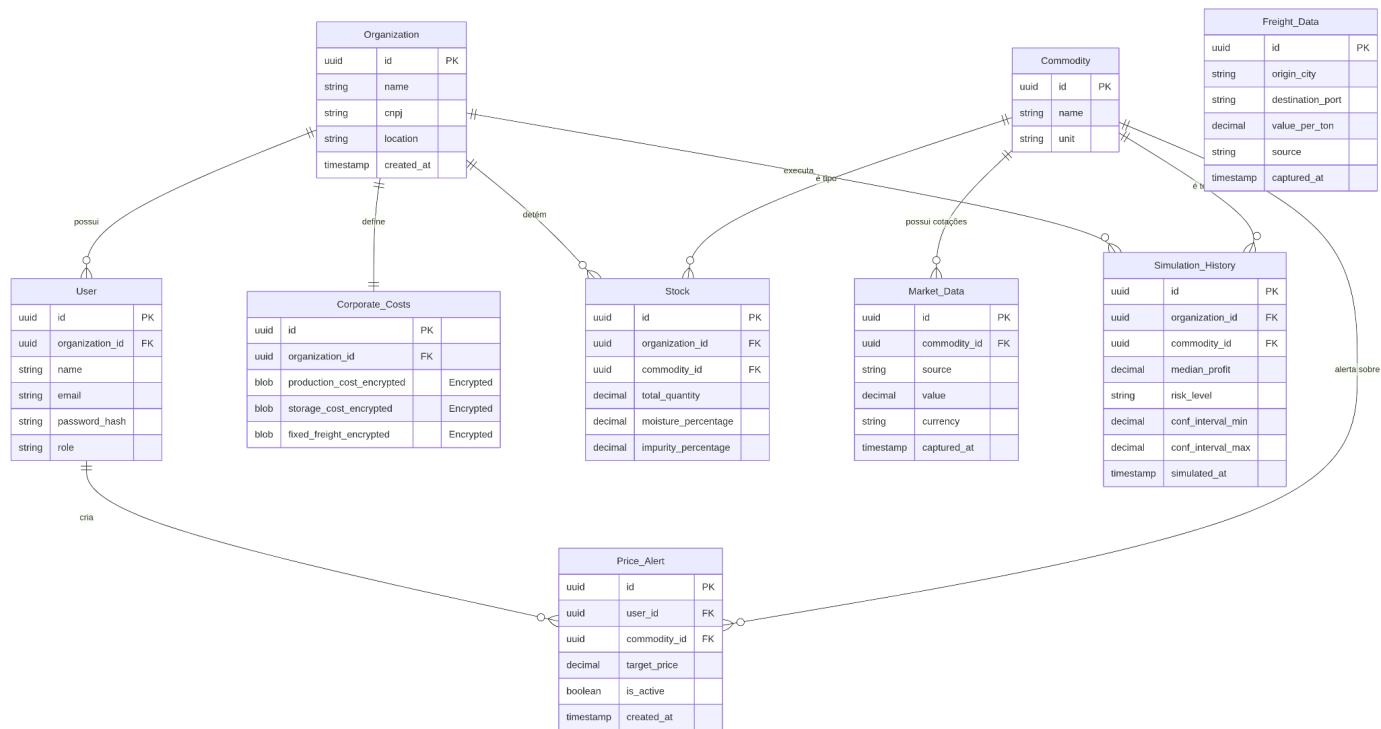
5.3 Inteligência de Mercado e Simulação

Estas entidades formam a base para o motor de **Monte Carlo** executado pelo serviço em Python:

- **Market_Data & Freight_Data:** Funcionam como repositórios de séries temporais. Armazenam o histórico de cotações das bolsas (CBOT) e fretes regionais (IMEA). O histórico de preços é o que permite ao sistema calcular a volatilidade necessária para as simulações probabilísticas.
- **Simulation_History:** Registra o resultado final de cada processamento pesado. Em vez de reprocessar 10.000 iterações toda vez que o usuário abre o dashboard, o sistema consulta esta tabela para exibir a última predição de lucro e o intervalo de confiança gerado.

5.4 Monitoramento e Notificações

- **Price_Alert:** Vincula o interesse do usuário a uma variação de preço. O sistema monitora a tabela **Market_Data** e, ao encontrar uma correspondência com os critérios da **Price_Alert**, aciona o fluxo de mensageria assíncrona.



6 - Estratégia de Implantação e DevOps

A infraestrutura é projetada para suportar a natureza distribuída do sistema (Java e Python), garantindo que a complexidade técnica não onere a operação financeira do projeto.

6.1 - Containerização e Orquestração (Docker)

Para assegurar a paridade entre os ambientes de desenvolvimento e produção, o sistema adota a tecnologia de containers.

- **Isolamento de Processos:** Cada serviço é encapsulado em uma imagem Docker independente. Isso permite que dependências específicas do Python não interfiram no ambiente de execução do Java.
- **Docker Compose:** A orquestração dos serviços é realizada via Docker Compose, permitindo que a stack completa (aplicações, banco de dados e cache) seja instanciada de forma atômica e coordenada.

- **Otimização de Recursos:** As imagens são construídas sobre distribuições Linux minimalistas, reduzindo o consumo de armazenamento e a superfície de vulnerabilidades.

6.2 - Gestão Evolutiva de Dados (Flyway)

A integridade do esquema de banco de dados é mantida através de migrações versionadas.

- **Automação de Schema:** O uso do **Flyway** integrado ao Spring Boot garante que todas as alterações estruturais no PostgreSQL sejam aplicadas de forma sequencial e controlada.
- **Rastreabilidade:** Cada alteração no banco de dados é tratada como código, permitindo o rastreio de versões e impedindo inconsistências entre o código da aplicação e a estrutura de tabelas existente.

6.3 - Entrega Mobile via Progressive Web App (PWA)

Considerando a necessidade de acesso em dispositivos móveis sem a complexidade de manutenção de lojas de aplicativos (App Store/Play Store), o sistema adota a estratégia de **PWA**.

- **Acessibilidade:** A interface web é otimizada para o comportamento nativo em dispositivos móveis, permitindo instalação direta via navegador, funcionamento em tela cheia e acesso rápido através da tela de início do usuário.
- **Desenvolvimento Mobile-First:** O design é projetado prioritariamente para telas reduzidas, garantindo usabilidade em campo.

6.4 - Ciclo de Vida e Sustentação Operacional

Eixo	Tecnologia / Técnica	Objetivo
Integração Contínua (CI)	GitHub Actions	Execução automatizada de testes (TDD) para validar a integridade dos cálculos a cada atualização.
Hospedagem	Virtual Private Server (VPS)	Provisionamento de infraestrutura de baixo custo com controle total sobre o ambiente.

Gateway / Proxy	Nginx	Gerenciamento de certificados SSL/HTTPS e roteamento de tráfego para os containers internos.
Saúde do Sistema	Spring Actuator	Monitoramento em tempo real do estado dos serviços e consumo de memória.
Gestão de Logs	SLF4J / Docker Logs	Registro centralizado de eventos para auditoria e diagnóstico de falhas operacionais.

6.5 - Padronização de Nomenclatura e Domínio

Para alinhar a documentação técnica com a linguagem de negócios do agronegócio, os termos internos são mapeados conforme a tabela abaixo:

Termo Técnico / Interno	Termo de Negócio (Interface)	Justificativa
<i>Simulation Service</i>	Motor de Tendências	Transmite a ideia de análise de mercado ativa.
<i>Monte Carlo Simulation</i>	Análise de Cenários de Lucro	Termo mais intuitivo e profissional para o produtor.
<i>Intelligence Schema</i>	Módulo de Predição	Reflete a finalidade estatística do armazenamento.
<i>Push Notification</i>	Alerta de Gatilho de Preço	Define claramente a ação e o motivo do contato.

7 - Planejamento de desenvolvimento do sistema

O planejamento macro do AgroBasis permanece como direcionador da evolução do produto, porém sua execução passa a adotar uma abordagem incremental baseada na construção de uma **espinha dorsal funcional do sistema**. Em vez de concluir grandes blocos isoladamente antes de validar o fluxo real da aplicação, a estratégia consiste em estruturar primeiro uma base mínima, executável e coerente de ponta a ponta, sobre a qual as demais capacidades serão progressivamente refinadas.

Essa abordagem reduz o risco de decisões prematuras, facilita a validação contínua da arquitetura e permite que a complexidade seja introduzida somente quando houver sustentação suficiente no domínio e na implementação existente.

Etapa A — Espinha dorsal funcional do domínio

A primeira etapa tem como objetivo estabelecer a base executável do sistema, contemplando os principais módulos e fluxos necessários para que o produto já seja capaz de representar, de forma mínima, o ciclo central de operação.

Nessa etapa, são consolidados:

- o ambiente local de desenvolvimento com Docker e PostgreSQL;
- o versionamento de banco com Flyway;
- a estrutura modular do **core-service**;
- os módulos de organização, identidade, estrutura produtiva, mercado, custos e pricing;
- os endpoints essenciais;
- os testes fundamentais de aplicação e persistência.

O resultado esperado é um fluxo funcional de ponta a ponta, permitindo, por exemplo:

- criar uma organização;
- cadastrar usuários vinculados à organização;
- cadastrar fazendas e talhões;
- associar commodities aos talhões;
- registrar cotações de mercado e taxas de câmbio;
- registrar custos base por commodity;
- consultar o preço atual calculado para uma organização.

Essa etapa representa a validação estrutural do sistema e serve como fundamento para toda a evolução posterior.

Etapa B — Segurança, isolamento e robustez da base

Uma vez estabelecida a espinha dorsal funcional, a segunda etapa concentra-se no endurecimento da base arquitetural, com foco em segurança, isolamento entre tenants e robustez dos fluxos já existentes.

Nessa etapa, são aprofundados:

- o modelo de multitenancy baseado em organização;
- os controles de acesso e identidade;
- a autenticação e autorização via Spring Security e JWT;
- o tratamento padronizado de erros;
- a consistência entre contratos de API, regras de negócio e persistência;
- a ampliação da cobertura de testes.

O objetivo dessa etapa é garantir que a base já construída opere com segurança e previsibilidade, sustentando a evolução do sistema sem comprometer integridade ou isolamento de dados.

Etapa C — Primeira entrega de valor comercial

Com a base estrutural, segura e economicamente funcional já consolidada, o sistema passa a evoluir do cálculo determinístico para uma camada analítica mais orientada à leitura e interpretação dos resultados.

Essa etapa foca em transformar o pricing, que já considera mercado, custo, frete e ajuste comercial, em uma visão mais útil para análise de negócio e comparação operacional.

São previstos:

- a consolidação do pricing como núcleo analítico da aplicação;
- a criação de visões comparativas entre componentes do cálculo, como preço convertido, preço ajustado, preço líquido e preço comercial;
- a ampliação da memória de cálculo e dos indicadores derivados;
- a possibilidade de consultas analíticas por organização, fazenda e commodity;
- a preparação da base conceitual para simulações e cenários futuros.

O objetivo dessa etapa é fazer o sistema deixar de responder apenas “qual é o valor atual?” e começar a responder também “como esse valor é composto e o que ele representa dentro da operação?”.

Etapa D — Simulação e cenários probabilísticos

Após a consolidação analítica do pricing, o sistema passa a incorporar o eixo de simulação e cenários, iniciando a camada de inteligência propriamente dita do produto.

Essa etapa corresponde à introdução do módulo analítico especializado, com foco em cenários futuros, sensibilidade e risco.

São previstos:

- a implantação do intelligence-service;

- a definição dos contratos entre core-service e intelligence-service;
- a introdução do ambiente Python com FastAPI;
- a implementação inicial do motor estatístico e das simulações;
- a construção de cenários com variação de preço, câmbio e componentes econômicos;
- a base para curvas probabilísticas, análise de sensibilidade e primeiros estudos de risco.

O objetivo dessa etapa é permitir que o sistema avance do cálculo do estado atual para a análise de cenários possíveis, agregando uma primeira camada de inteligência orientada à decisão.

Etapa E — Alertas, gatilhos e apoio operacional à decisão

Com a base analítica e de simulação amadurecida, torna-se viável introduzir mecanismos de monitoramento e reação a condições relevantes para o negócio.

Essa etapa foca em transformar o sistema em um apoio mais ativo à operação, permitindo que ele não apenas informe e simule, mas também monitore situações importantes e sinalize oportunidades ou riscos.

São previstos:

- o módulo de regras de alerta;
- a configuração de gatilhos por organização, fazenda e commodity;
- o monitoramento de preço comercial, margens e condições configuradas;
- a base para notificações e alertas futuros;
- a evolução do sistema de leitura passiva para acompanhamento ativo de eventos econômicos.

O objetivo dessa etapa é permitir que o sistema responda não apenas “quanto vale” ou “o que pode acontecer”, mas também “quando vale a pena agir”.

Etapa F — Experiência do usuário, operação e produção

A etapa final concentra-se na consolidação do sistema para uso real, com foco em experiência, automação operacional e disponibilidade em produção.

São previstos:

- refinamento da experiência mobile-first;
- evolução do frontend e da legibilidade das análises;
- construção das primeiras visualizações orientadas a decisão;
- configuração de pipeline de CI/CD;
- provisionamento da infraestrutura com Terraform;
- publicação em ambiente de produção;
- configuração de Nginx e HTTPS;
- estabelecimento dos primeiros mecanismos de operação contínua.

O objetivo dessa etapa é transformar a base técnica, econômica e analítica já consolidada em um produto utilizável, operável e apto para evolução contínua.

Considerações metodológicas

O plano acima deve ser entendido como um **roadmap macro de evolução**, e não como uma sequência rígida de entregas monolíticas. Sua função é orientar a direção do produto e organizar o crescimento do sistema em grandes frentes de desenvolvimento.

A execução prática, no entanto, seguirá uma lógica incremental e iterativa. Cada etapa será desdobrada em fatias menores e executáveis, sempre priorizando a validação de fluxos reais do sistema antes da introdução de novas camadas de complexidade.

Além desse plano macro, será mantida uma documentação técnica complementar contendo:

- decisões arquiteturais;
- justificativas de modelagem;
- detalhamento dos módulos;
- convenções de implementação;
- critérios de testes;
- e observações sobre limitações e evoluções futuras.

Essa combinação entre **planejamento macro** e **execução incremental orientada por espinha dorsal funcional** estabelece uma base mais segura para o desenvolvimento do AgroBasis, conciliando clareza arquitetural, aprendizado progressivo e redução de risco técnico.

8 - Documentação técnica do andamento do projeto

Esta seção registra o estado técnico efetivamente implementado no projeto durante a evolução do sistema. Seu objetivo é documentar, de forma incremental, as decisões de modelagem, as responsabilidades dos módulos e as justificativas arquiteturais adotadas ao longo do desenvolvimento.

Diferentemente do roadmap macro, que define a direção evolutiva do produto em alto nível, esta documentação técnica descreve o que foi efetivamente construído, como os módulos foram organizados e quais simplificações de domínio foram assumidas na implementação atual.

Documentação técnica da Etapa A — Espinha dorsal funcional do domínio

Na etapa inicial, o sistema foi estruturado como um **monólito modular**, com o objetivo de permitir uma implementação mais simples, coesa e evolutiva do domínio, sem introduzir prematuramente a complexidade operacional de um sistema distribuído.

Essa decisão foi tomada com base em dois critérios principais:

- necessidade de consolidar primeiro a modelagem de domínio central do produto;
- redução da complexidade de desenvolvimento e manutenção em uma fase ainda exploratória do sistema.

A arquitetura inicial foi organizada em dois grandes blocos:

- um módulo de infraestrutura local para suporte ao ambiente de desenvolvimento;
- um núcleo de domínio concentrado no `core-service`.

Infraestrutura local

No nível de infraestrutura, foi configurado o ambiente de desenvolvimento com containers Docker, inicialmente contendo:

- PostgreSQL, como banco de dados principal do sistema;
- Redis, preparado para futuras responsabilidades relacionadas a cache, integração e processamento assíncrono.

Nesse estágio, o PostgreSQL já está em uso ativo no sistema, enquanto o Redis permanece provisionado como parte da base infraestrutural, mas ainda sem papel central no fluxo funcional da Etapa A.

Núcleo do domínio (`core-service`)

O `core-service` foi estruturado como o núcleo modular do sistema e organizado em módulos de domínio distintos, cada um responsável por um conjunto específico de responsabilidades de negócio.

Os módulos atualmente implementados são:

- `organization`
- `identity`
- `farm`
- `market`
- `cost`
- `pricing`

Além disso, existe um módulo `shared`, responsável por elementos transversais, como configuração, documentação da API e tratamento global de erros.

Módulo `organization`

O módulo `organization` representa a entidade central do sistema do ponto de vista organizacional. Ele é responsável por definir o tenant da aplicação e, portanto, constitui a base do modelo multitenant adotado.

No estado atual do sistema:

- todos os dados relevantes pertencem logicamente a uma organização;
- usuários existem dentro de uma organização;
- fazendas, talhões, custos e cálculos econômicos são associados à organização;
- o isolamento entre empresas é modelado a partir dessa entidade.

Assim, a **Organization** não é apenas uma entidade cadastral, mas a principal fronteira lógica de dados do sistema.

Módulo **identity**

O módulo **identity** é responsável pela representação dos usuários e seus papéis no sistema.

Seu papel atual é:

- modelar a existência do usuário dentro da organização;
- sustentar futuras regras de autenticação e autorização;
- preparar a base para o uso posterior de Spring Security e JWT.

Os usuários foram modelados como entidades pertencentes a uma organização, o que mantém consistência com o modelo multitenant adotado.

No estágio atual, o módulo já contém:

- entidade de usuário;
- enumeração de papéis de acesso;
- endpoints básicos;
- persistência;
- testes de aplicação e integração.

Módulo **farm**

O módulo **farm** representa a **estrutura produtiva física básica** da organização.

Ele foi modelado com três conceitos principais:

- **Farm**
- **Plot**
- **Commodity**

Farm

A entidade **Farm** representa a unidade produtiva macro da organização. Sua função atual é identificar a fazenda como local físico de produção, incluindo atributos como localização e área total.

Mesmo que parte dessas informações também esteja relacionada ao talhão, a fazenda não é redundante: ela representa um nível estrutural superior e serve como agrupador lógico e operacional dos talhões.

Plot

A entidade **Plot** representa a subdivisão operacional da fazenda. Ela delimita a área produtiva em um nível mais granular e permite associar, no estado atual do sistema, uma commodity principal ao talhão.

Commodity

A **Commodity** foi modelada como **enum**, por se tratar de um conjunto fechado de produtos dentro do escopo inicial do sistema. Essa escolha simplifica a modelagem, evita inconsistência semântica no cadastro e favorece a integração com os módulos de mercado e pricing.

Limitação conhecida

O módulo **farm** modela atualmente a estrutura física básica da produção, mas **ainda não modela a produção temporal do talhão**. Isso significa que:

- a commodity associada ao talhão representa o produto atual ou principal;
- o sistema ainda não representa formalmente ciclos de produção, safras ou histórico produtivo da área.

Essa é uma limitação conhecida da etapa atual e está prevista como ponto de evolução futura do domínio.

Módulo **market**

O módulo **market** foi criado para representar e armazenar **dados externos de referência econômica**.

Seu papel não é calcular valor para a empresa, mas disponibilizar as informações de mercado que servirão como entrada para os cálculos econômicos do sistema.

Atualmente, ele foi modelado com dois conceitos:

- **MarketQuote**
- **ExchangeRate**

MarketQuote

Representa uma cotação de mercado de determinada commodity, incluindo:

- produto cotado;
- valor;
- moeda;

- unidade;
- fonte;
- horário da cotação.

ExchangeRate

Representa uma taxa de câmbio entre moedas, incluindo:

- moeda de origem;
- moeda de destino;
- taxa;
- fonte;
- horário da cotação.

O módulo já está estruturado com:

- entidades;
- migrations;
- endpoints básicos;
- services;
- testes de aplicação;
- testes de persistência.

Essa base prepara o sistema para futura integração com APIs externas de mercado e câmbio.

Módulo **cost**

O módulo **cost** foi criado para representar os **custos internos de referência** da organização.

Na etapa atual, ele ainda não modela custos por safra, por fazenda ou por talhão. Em vez disso, foi adotado um modelo simplificado e funcional, baseado em um perfil de custo por:

- organização
- commodity

Seu conceito central é o **CostProfile**, que representa o custo base em BRL por tonelada daquela commodity para a organização.

Essa decisão foi adotada porque:

- permite incorporar custo ao cálculo econômico desde cedo;
- evita depender, prematuramente, de uma modelagem mais complexa de produção temporal;
- aproxima o sistema de uma lógica útil para pricing sem inflar desnecessariamente o domínio.

O módulo contém:

- entidade **CostProfile**;
- constraint de unicidade por organização e commodity;
- endpoints básicos de criação, busca, listagem e atualização;
- testes de aplicação e persistência.

Módulo **pricing**

O módulo **pricing** foi criado para transformar os dados externos do mercado e os dados internos de custo em **informação econômica útil para a organização**.

Ele não atua como um simples cadastro de valores, mas como um módulo de cálculo e interpretação econômica.

Na primeira versão implementada, o **pricing** realiza:

- a busca da cotação mais recente da commodity;
- a busca da taxa de câmbio mais recente de USD para BRL;
- a validação do contexto de cálculo suportado;
- o cálculo do preço convertido em BRL por tonelada;
- a montagem de uma memória de cálculo explicável.

A resposta do módulo inclui:

- resultado do cálculo;
- snapshots da cotação e do câmbio utilizados;
- memória de cálculo detalhada;
- horário da geração da análise.

Essa abordagem foi escolhida para garantir rastreabilidade, auditabilidade e clareza dos valores retornados pelo sistema.

Evolução imediata prevista

A próxima evolução do módulo **pricing** consiste em incorporar o **CostProfile** ao cálculo, permitindo retornar não apenas o preço convertido, mas também o preço ajustado pelo custo base da organização.

Com isso, o módulo deixará de responder apenas “quanto vale em moeda local” e passará a responder “qual o valor econômico atual para esta organização, dado o seu custo configurado”.

Estrutura de testes

Durante a Etapa A, foi adotada uma estratégia de testes baseada principalmente em:

- testes de aplicação (**service tests**);
- testes de persistência (**repository integration tests**);
- alguns testes de fluxo HTTP mais amplos, quando úteis.

A prioridade foi dada a testes de caso de uso e persistência, por oferecerem maior valor para a estabilidade do domínio e da lógica de negócio. Testes isolados de controller não foram adotados como foco principal nesta etapa.

Essa estratégia permitiu validar:

- comportamento dos módulos;
- regras de criação, consulta e atualização;
- integridade de persistência;
- consistência do fluxo econômico inicial.

Estado final da Etapa A

Ao final da Etapa A, o sistema já possui uma espinha dorsal funcional capaz de executar, de forma integrada, o seguinte fluxo mínimo:

1. criação de organização;
2. criação de usuário vinculado à organização;
3. criação de fazenda;
4. criação de talhão com commodity;
5. cadastro de cotação de mercado;
6. cadastro de taxa de câmbio;
7. cadastro de perfil de custo;
8. cálculo do preço atual por commodity.

Esse fluxo caracteriza a conclusão funcional da espinha dorsal do sistema e estabelece a base para as próximas evoluções do produto.

Documentação técnica da Etapa B — Segurança, isolamento e robustez da base

Na Etapa B, o sistema evoluiu da condição de espinha dorsal funcional para uma base operacional mais segura, mais robusta e economicamente mais útil.

Se a Etapa A teve como foco principal consolidar a modelagem central do domínio e estabelecer o fluxo mínimo funcional do produto, a Etapa B teve como objetivo aprofundar quatro dimensões fundamentais do sistema:

- segurança e autenticação;
- isolamento real entre organizações;
- integração com dados externos de mercado;
- refinamento progressivo do cálculo econômico.

Essa etapa foi dividida em quatro sub etapas:

- B.1 — Segurança e Acesso
- B.2 — Integração externa de mercado
- B.3 — Evolução econômica do pricing com frete
- B.4 — Refinamento comercial do pricing

A divisão em sub etapas foi necessária porque o escopo técnico e de negócio da Etapa B passou a abranger responsabilidades muito diferentes, que precisavam ser implementadas de forma incremental, validável e coerente com a maturidade atual do projeto.

B.1 — Segurança e Acesso

A subetapa B.1 teve como objetivo transformar a identidade modelada no domínio em um mecanismo real de autenticação, autorização e controle de acesso no backend.

Até o fim da Etapa A, o sistema já possuía usuários, papéis de acesso e associação lógica à organização, mas ainda não aplicava, no fluxo HTTP, mecanismos efetivos de segurança. A B.1 foi a etapa em que essa modelagem passou a operar de forma concreta na aplicação.

Autenticação e credenciais

Foi adotado Spring Security com autenticação baseada em JWT, tornando o backend stateless e adequado ao modelo de API do sistema.

As principais decisões técnicas dessa frente foram:

- uso de BCrypt para hash de senha;
- emissão de JWT com duração de 12 horas;
- criação de um principal customizado para representar o usuário autenticado no contexto de segurança;
- validação de token via filtro antes da execução dos endpoints protegidos.

Com isso, o sistema passou a autenticar usuários por login e senha e a carregar, no contexto autenticado, informações necessárias para autorização e tenant enforcement, como:

- identificador do usuário;
- identificador da organização;
- papel de acesso;
- email.
- Modelo de acesso organizacional

A B.1 também consolidou uma decisão importante de domínio: identidade e vínculo organizacional não são a mesma coisa.

Foi adotado o seguinte modelo:

- o usuário pode criar uma conta no sistema;
- a conta nasce sem acesso efetivo à organização;
- o usuário solicita vínculo com uma organização;
- um administrador dessa organização aprova ou rejeita a solicitação;
- somente após aprovação o usuário passa a pertencer ao tenant e pode operar normalmente no sistema.

Para suportar esse fluxo, o módulo identity foi expandido com:

- status de acesso da conta;
- solicitação de vínculo organizacional;
- aprovação e rejeição de vínculo;
- controle de acesso por papel.

O status de acesso da conta foi modelado com três estados:

1. PENDING_ORGANIZATION_APPROVAL
2. ACTIVE
3. REJECTED

Essa escolha permitiu representar com clareza a diferença entre:

- existência da conta;
- permissão para autenticar;
- vínculo efetivo com a organização.

Tenant enforcement inicial

Além da autenticação e autorização, a B.1 introduziu os primeiros mecanismos reais de tenant enforcement.

Na prática, isso significa que o sistema passou a verificar se o organizationId informado em determinados fluxos sensíveis corresponde de fato à organização do usuário autenticado.

Esse controle foi aplicado inicialmente em fluxos mais críticos, como:

- cost
- pricing
- aprovação e listagem de solicitações de vínculo

Essa abordagem foi escolhida porque permitia começar a aplicar isolamento organizacional real sem exigir, ainda nessa etapa, a remoção imediata de todos os parâmetros organizacionais da API.

Testes da B.1

A B.1 foi validada com uma combinação de:

- testes de aplicação;
- testes de persistência; criação de solicitação de vínculo;
- aprovação e rejeição por administrador;
- tentativa de acesso sem token;
- tentativa de acesso com token inválido;
- tentativa de acesso com role insuficiente;
- tentativa de acesso a dados de outro tenant.

Ao final da B.1, o sistema deixou de ter apenas identidade modelada no domínio e passou a ter segurança real aplicada ao backend.

B.2 — Integração externa de mercado

A subetapa B.2 teve como objetivo transformar o módulo market de um módulo apenas cadastral em um módulo de ingestão controlada de referências externas reais.

Até esse momento, o sistema já conseguia armazenar cotações e taxas de câmbio, mas dependia de cadastro manual desses dados. A B.2 foi a etapa em que o sistema passou a obter informações externas e persisti-las com histórico.

Estratégia adotada

A estratégia escolhida para a B.2 foi a de sincronização manual via endpoint administrativo.

Isso significa que:

- o sistema ainda não atualiza automaticamente os dados de mercado;
- o pricing não chama APIs externas diretamente;
- a sincronização é disparada manualmente por um endpoint protegido;
- os dados externos são persistidos no banco;
- o pricing continua consumindo apenas os registros persistidos.
- testes HTTP de segurança;

Testes de tenant enforcement.

Esses testes passaram a cobrir, entre outros cenários:

- login com sucesso;
- credenciais inválidas;
- usuário pendente;
- usuário rejeitado;

Essa escolha foi importante para manter a fase simples, testável e previsível.

Integração de câmbio

Para câmbio, foi implementado um client concreto de integração com o BCB/PTAX.

Esse client ficou responsável por:

- consultar a fonte externa;
- interpretar a resposta;
- normalizar os dados para um DTO interno;

- devolver a taxa ao serviço de sincronização.

A B.2 adotou como fluxo principal o par:

- USD -> BRL

O par inverso também passou a ser derivável a partir da mesma cotação, mas o uso principal do sistema continua orientado ao cálculo de pricing em moeda local a partir de cotações em dólar.

Integração de commodity

Para cotação de commodity, a modelagem adotou uma decisão arquitetural importante: abstração de provider.

Em vez de acoplar diretamente o restante do sistema à implementação concreta da fonte, foi introduzida uma interface responsável por representar um provider de cotação de commodity.

A implementação inicial foi feita com uma integração voltada à B3, encapsulando:

- acesso à fonte;
- leitura da resposta;
- normalização dos dados externos;
- devolução de um DTO interno estável para o restante do módulo market.

Essa escolha preserva o domínio do sistema caso a fonte externa precise mudar no futuro.

Serviço de sincronização

Foi criado um orquestrador específico para a sincronização externa, responsável por:

- chamar os clients;
- validar os dados recebidos;
- converter o resultado em entidade do domínio;
- persistir novo histórico;
- retornar os DTOs públicos da aplicação.

Essa separação foi importante para manter o market organizado em três níveis:

- integração externa;
- persistência interna;
- exposição HTTP.

Persistência histórica.

Uma decisão importante da B.2 foi manter o modelo histórico do módulo market.

Cada sincronização bem-sucedida gera um novo registro em:

- market_quote
- exchange_rate

Essa estratégia foi mantida porque garante:

- rastreabilidade;
- auditabilidade;
- reprodutibilidade do cálculo;
- possibilidade de evolução futura para análise temporal.

Testes da B.2

A B.2 foi validada por meio de:

- testes do MarketSyncService;
- testes HTTP dos endpoints de sincronização;
- atualização do fluxo de backbone para incluir sincronização e cálculo com dados persistidos.

Os testes passaram a cobrir:

- sincronização com sucesso;
- falha externa;
- dado externo inválido;
- commodity não suportada;
- par cambial não suportado;
- acesso sem token;
- acesso com role insuficiente;
- fluxo real com persistência e uso posterior pelo pricing.

Ao final da B.2, o sistema deixou de depender apenas de cadastro manual de mercado e passou a operar com dados externos reais persistidos no banco.

B.3 — Evolução econômica do pricing com frete

A subetapa B.3 teve como objetivo evoluir o cálculo econômico do sistema, aproximando o pricing da realidade operacional.

Até esse ponto, o sistema já calculava:

- preço convertido em BRL por tonelada;
- preço ajustado pelo custo base.

A B.3 introduziu um novo componente econômico:

- frete.

Decisão de modelagem

O frete foi modelado por:

- organização
- fazenda
- commodity

Essa escolha foi feita porque:

- a organização continua representando o tenant e o dono lógico do dado;
- a fazenda representa a origem física da operação;
- a commodity representa o produto transportado.

Assim, o frete passou a ser tratado como um dado interno da organização, mas com vínculo explícito à origem produtiva da operação.

FreightProfile

Foi criado o conceito de FreightProfile, representando o frete base em BRL por tonelada para uma combinação de:

- organização;
- fazenda;
- commodity.

Esse perfil foi modelado com:

- entidade própria;
- migration dedicada;
- constraint de unicidade;
- validação de não-negatividade;
- endpoints de criação, consulta, listagem e atualização;
- testes de aplicação e persistência.

A não-negatividade do frete foi protegida em três níveis:

- validação da entrada;
- validação da entidade;
- constraint no banco.

Evolução do pricing

Com a introdução do frete, o pricing passou a calcular três níveis de valor:

- convertedPrice
- adjustedPrice
- netPrice

As fórmulas passaram a ser:

- $\text{convertedPrice} = \text{marketQuote.price} \times \text{exchangeRate.rate}$

- $\text{adjustedPrice} = \text{convertedPrice} - \text{costPerTon}$
- $\text{netPrice} = \text{adjustedPrice} - \text{freightPerTon}$

Além do novo cálculo, o retorno do pricing foi expandido para incluir:

- `farmId`
- `freightPerTon`
- `netPrice`
- memória de cálculo detalhada do novo passo

Testes da B.3

A B.3 foi validada com:

- testes do `FreightProfileService`;
- testes do `FreightProfileRepository`;
- atualização dos testes do `PricingService`;
- atualização do `CoreBackboneFlowIT`.

Os cenários passaram a cobrir:

- criação de perfil de frete;
- validação de fazenda pertencente à organização;
- duplicidade;
- ausência de frete;
- cálculo de `netPrice`;
- fluxo ponta a ponta com frete integrado ao pricing.

Ao final da B.3, o sistema passou a entregar um preço líquido preliminar, economicamente mais próximo da operação real.

B.4 — Refinamento comercial do pricing

A subetapa B.4 teve como objetivo adicionar uma camada comercial simples sobre o cálculo econômico já existente.

Até esse momento, o sistema já conseguia retornar:

- preço convertido;
- preço ajustado por custo;
- preço líquido após frete.

A B.4 introduziu um novo conceito:

- ajuste comercial.

Decisão de modelagem

Assim como o frete, o ajuste comercial foi modelado por:

- organização
- fazenda
- commodity

Essa escolha foi feita para manter consistência com o restante do domínio econômico da operação e evitar a introdução prematura de um modelo comercial mais complexo.

Nesta fase, o ajuste comercial foi modelado como:

- abatimento fixo em BRL por tonelada

Isso significa que ele:

- não é percentual;
- não possui múltiplos tipos;
- não representa ainda toda a complexidade comercial do negócio;
- atua como uma camada determinística e simples de abatimento sobre o preço líquido.
- `CommercialAdjustmentProfile`

Foi criado o conceito de `CommercialAdjustmentProfile`, representando um ajuste comercial fixo por tonelada para a combinação de:

- organização;
- fazenda;
- commodity.

Esse novo perfil recebeu:

- entidade própria;
- migration dedicada;
- constraint de unicidade;
- validação de não-negatividade;
- endpoints de criação, consulta, listagem e atualização;
- testes de aplicação e persistência.

Assim como no frete, a regra de não-negatividade foi protegida em múltiplos níveis.

Evolução do pricing

Com a introdução do ajuste comercial, o pricing passou a calcular um quarto nível de valor:

- `commercialPrice`

As fórmulas passaram a ser:

- $\text{convertedPrice} = \text{marketQuote.price} \times \text{exchangeRate.rate}$
- $\text{adjustedPrice} = \text{convertedPrice} - \text{costPerTon}$
- $\text{netPrice} = \text{adjustedPrice} - \text{freightPerTon}$
- $\text{commercialPrice} = \text{netPrice} - \text{adjustmentPerTon}$

O retorno do pricing foi ampliado para incluir:

- adjustmentPerTon
- commercialPrice
- memória de cálculo do passo comercial

Essa evolução tornou o cálculo mais próximo da leitura econômica e comercial que a organização pode fazer sobre a operação.

Testes da B.4

A B.4 foi validada com:

- testes do CommercialAdjustmentProfileService;
- testes do CommercialAdjustmentProfileRepository;
- atualização dos testes do PricingService;
- atualização dos testes de segurança HTTP;
- atualização do CoreBackboneFlowIT.

Os testes passaram a cobrir:

- criação de ajuste comercial;
- validação de tenant e pertencimento da fazenda;
- duplicidade;
- ausência de ajuste comercial;
- cálculo de commercialPrice;
- fluxo ponta a ponta com preço comercial final.

Ao final da B.4, o sistema passou a ser capaz de retornar não apenas o valor econômico operacional, mas também um preço comercial ajustado, ainda de forma simples, determinística e explicável.

Estrutura de testes da Etapa B

Ao longo da Etapa B, a estratégia de testes adotada foi ampliada em relação à Etapa A.

Além de testes de aplicação e persistência, a etapa passou a incorporar com maior relevância:

- testes HTTP de segurança;
- testes de tenant enforcement;
- testes de backbone ponta a ponta;
- testes de integração entre persistência, segurança e cálculo econômico.

Essa estratégia foi importante porque a Etapa B introduziu preocupações que não existiam na Etapa A, como:

- autenticação e autorização;
- isolamento real entre organizações;
- integração com fontes externas;
- evolução encadeada do cálculo econômico.

Com isso, os testes passaram a validar não apenas módulos isolados, mas também o comportamento do sistema como produto integrado.

Estado final da Etapa B

Ao final da Etapa B, o sistema já é capaz de executar, de forma segura e integrada, um fluxo significativamente mais maduro do que o da Etapa A.

Esse fluxo inclui:

- criação de conta de usuário;
- solicitação de vínculo organizacional;
- aprovação administrativa do vínculo;
- autenticação com JWT;
- controle de acesso por role;
- tenant enforcement inicial;
- criação de organização, fazenda e talhão;
- sincronização ou cadastro de cotação de mercado;
- sincronização ou cadastro de taxa de câmbio;
- cadastro de perfil de custo;
- cadastro de perfil de frete;
- cadastro de perfil de ajuste comercial;
- cálculo do preço comercial final da commodity para uma determinada fazenda.

Com isso, a Etapa B conclui a transição do sistema de uma base funcional inicial para uma base operacional segura, integrada ao mercado e economicamente mais útil para o produto.

Documentação técnica da Etapa C — Segurança, isolamento e robustez da base